

```
import pandas as pd
import numpy as np

df=pd.read_csv("/content/mobile_sales_data.csv")

display(df.head())
```

	OrderID	Date	Product	Region	Quantity	UnitPrice	CostPerUnit	Disco
0	Ord001	01-01-2026	iPhone 14	Chennai	2.0	50000	42000.0	
1	Ord002	02-01-2026	Samsung S23	Hyderabad	3.0	20000	15000.0	
2	Ord003	03-01-2026	OnePlus 11	Bengaluru	4.0	30000	22000.0	
3	Ord004	04-01-2026	Redmi Note 13	Mumbai	NaN	15000	NaN	
4	Ord005	05-01-2026	Realme Narzo 60	Delhi	1.0	12000	8000.0	

Handling Missing Values

```
# Numeric columns -> median imputation
num_cols = ["Quantity", "UnitPrice", "CostPerUnit", "DiscountRate", "RAM", "Storage"]
for col in num_cols:
    df[col] = df[col].fillna(df[col].median())

# Categorical columns (general) -> mode imputation
cat_cols = ["Product", "Region", "Network"]
for col in cat_cols:
    df[col] = df[col].fillna(df[col].mode()[0])

binary_cols = ["Shipped", "Delivered"]

# Handling 'Shipped' column missing values and type conversion
df['Shipped'] = df['Shipped'].replace({0: 'No', 1: 'Yes', True: 'Yes', False: 'No'})
df['Shipped'] = df['Shipped'].replace('nan', np.nan)

shipped_mode_series = df['Shipped'].mode()
if not shipped_mode_series.empty:
    shipped_mode = shipped_mode_series.iloc[0]
else:
```

```

shipped_mode = 'No'

df['Shipped'] = df['Shipped'].fillna(shipped_mode)

# Handling 'Delivered' column missing values and type conversion
df['Delivered'] = df['Delivered'].replace({0: 'No', 1: 'Yes', True: 'Yes', False: 'No'})
df['Delivered'] = df['Delivered'].replace('nan', np.nan)
delivered_nan_mask = df['Delivered'].isna()
df.loc[delivered_nan_mask, 'Delivered'] = df.loc[delivered_nan_mask, 'Shipped']

```

Encoding catogorical features

```

# Binary Encoding
binary_cols = ["Shipped", "Delivered"]

for col in binary_cols:
    df[col] = df[col].astype(str).str.strip()
    df[col] = df[col].map({"Yes": 1, "No": 0})
    df[col] = df[col].fillna(0).astype(int)

display(df.head())

```

	OrderID	Date	Product	Region	Quantity	UnitPrice	CostPerUnit	Discount
1	Ord002	02-01-2026	Samsung S23	Hyderabad	0.164399	-0.502966	-0.353894	
2	Ord003	03-01-2026	OnePlus 11	Bengaluru	0.986394	0.321568	0.313831	-
6	Ord007	07-01-2026	Samsung A54	Hyderabad	0.164399	-0.090699	-0.067726	-
7	Ord008	08-01-2026	OnePlus Nord 3	Bengaluru	0.164399	0.156661	-0.258505	
9	Ord010	10-01-2026	Realme 11 Pro	Delhi	-1.479591	-0.338059	-0.258505	-

Descriptive Statistics

```

print(" descriptive statistics ")
df.describe()

```

descriptive statistics							
	Quantity	UnitPrice	CostPerUnit	DiscountRate	Shipped	Delivered	
count	27.000000	27.000000	27.000000	27.000000	27.0	27.0	27
mean	0.012178	-0.069322	-0.127786	0.020978	0.0	0.0	0
std	0.823165	0.487046	0.293590	1.051541	0.0	0.0	0
min	-1.479591	-1.162593	-1.021619	-1.255914	0.0	0.0	0
25%	0.164399	-0.338059	-0.258505	-1.255914	0.0	0.0	0
50%	0.164399	-0.090699	-0.258505	-0.024626	0.0	0.0	0
75%	0.164399	0.156661	0.123052	1.206662	0.0	0.0	0
max	0.986394	1.558369	0.313831	1.206662	0.0	0.0	0

Feature Scaling: Normalization and Standardization

```

from sklearn.preprocessing import StandardScaler

# Identify numerical columns for scaling (excluding binary and categorical)
# Re-using num_cols from previous steps for consistency
num_cols_to_scale = ["Quantity", "UnitPrice", "CostPerUnit", "DiscountRate",

# Initialize StandardScaler
scaler = StandardScaler()

# Apply standardization to the numerical columns
df[num_cols_to_scale] = scaler.fit_transform(df[num_cols_to_scale])

print("DataFrame after scaling numerical features:")
display(df.head())

```

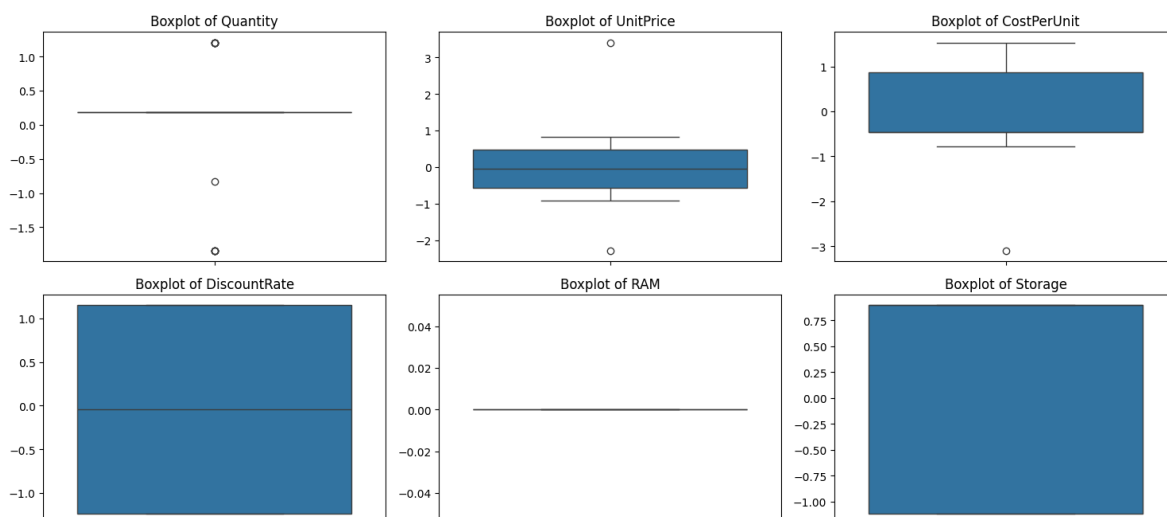
DataFrame after scaling numerical features:

	OrderID	Date	Product	Region	Quantity	UnitPrice	CostPerUnit	Disco
1	Ord002	02-01-2026	Samsung S23	Hyderabad	0.188445	-0.907316	-0.784818	
2	Ord003	03-01-2026	OnePlus 11	Bengaluru	1.206045	0.817862	1.532848	-
6	Ord007	07-01-2026	Samsung A54	Hyderabad	0.188445	-0.044727	0.208467	-
7	Ord008	08-01-2026	OnePlus Nord 3	Bengaluru	0.188445	0.472827	-0.453723	
9	Ord010	10-01-2026	Realme 11 Pro	Delhi	-1.846757	-0.562280	-0.453723	-

Outlier Detection and Removal

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(15, 10))
for i, col in enumerate(num_cols_to_scale):
    plt.subplot(3, 3, i + 1)
    sns.boxplot(y=df[col])
    plt.title(f'Boxplot of {col}')
    plt.ylabel('')
plt.tight_layout()
plt.show()
```



```
def remove_outliers_iqr(df, column):
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    df_cleaned = df[(df[column] >= lower_bound) & (df[column] <= upper_bound)]
    return df_cleaned

# Apply the outlier removal function to each numerical column
original_rows = df.shape[0]
print(f"Original DataFrame shape: {df.shape}")

for col in num_cols_to_scale:
    df = remove_outliers_iqr(df, col)

print(f"DataFrame shape after outlier removal: {df.shape}")
print(f"Number of rows removed: {original_rows - df.shape[0]}")
display(df.head())
```

Original DataFrame shape: (15, 13)
 DataFrame shape after outlier removal: (15, 13)
 Number of rows removed: 0

	OrderID	Date	Product	Region	Quantity	UnitPrice	CostPerUnit	Discount
1	Ord002	02-01-2026	Samsung S23	Hyderabad	0.188445	-0.907316	-0.784818	
6	Ord007	07-01-2026	Samsung A54	Hyderabad	0.188445	-0.044727	0.208467	
7	Ord008	08-01-2026	OnePlus Nord 3	Bengaluru	0.188445	0.472827	-0.453723	
11	Ord012	12-01-2026	Samsung S23	Hyderabad	0.188445	-0.907316	-0.784818	
16	Ord017	02-02-2026	Samsung A54	Hyderabad	0.188445	-0.044727	0.208467	

```
#AFTER REMOVING OUTLIERS
display(df)
```